

第六组模型审计报告

张子岳2022201596 郝文馨2023200073

0. 引言

- 1. 已经运行代码，确保代码可运行。
- 2. 模型采用了有效手段处理缺失值（中位数填补等）和异常值（log变换，缩尾等），但这一过程有数据泄露风险。
- 3. 模型有着较大的多重共线性风险，这影响了模型的稳健性。

1. 数据预处理

1.1 特征删除的合理性

一些列被删除时缺乏充分依据。例如，开发商、物业公司和车位。（原数据集中是”停车位“，所以应该没有成功drop）。车位对价格有预测能力，尤其是可以用车位/房屋总数求出户均车位数。rent里面”供水“”供暖“”供电“也并非文本描述列，而是类别变量。这可能损失对房价有影响的重要信息。并且，如果这种判断是基于整个数据集（包含测试）的分布得出的，理论上也属于信息泄漏的一种。

建议：在删除特征前可以再进行一些检验，现在只看了缺失值。可以尝试补充：相关性检验等。另外，对于price和rent，如果同一列有不同的处理，也要有充分的理由（比如售价数据中的装修情况被独热编码，而租金数据中的装修列却被直接删除）。

a. 删除无用和信息冗余的列

```
cols_to_drop_price = [
    '抵押信息',      # 完全为空
    '别墅类型',      # 缺失值超过98%
    '物业办公电话',  # 对价格无预测能力
    '区县',          # 与'区域'列信息高度重叠
    '开发商',
    '物业公司',
    '环线位置',      # 与'环线'列信息高度重叠
    'coord_x',       # 与 Lon/Lat 重叠
    'coord_y',       # 与 Lon/Lat 重叠
    # 文本描述列
    '房屋优势', '核心卖点', '户型介绍', '周边配套', '交通出行', '客户反馈'
]

df_price = df_price.drop(columns=cols_to_drop_price, errors='ignore')

cols_to_drop_rent = [
    '车位',          # 对价格无预测能力
    '开发商',        # 与'区域'列信息高度重叠
    '物业公司',
    '物业办公电话',  # 与'环线'列信息高度重叠
    'coord_x',       # 与 Lon/Lat 重叠
    'coord_y',       # 与 Lon/Lat 重叠
    # 文本描述列
    '客户反馈',
    '装修',
    '供水',
    '供暖',
    '供电',
]

df_rent = df_rent.drop(columns=cols_to_drop_rent, errors='ignore')
```

1.2 缺失值处理带来的数据泄露风险

部分缺失值填充可能使用了测试集的信息。例如，对房屋户型提取的室/厅/卫数进行缺失填充时，直接计算了整合集（训练+测试）的中位数。这在理论上将测试集分布泄漏给训练过程。

```
if col in df.columns:
    median_val = df[col].median()
    df[col].fillna(median_val, inplace=True)
```

建议：严格使用训练集计算填充值，再将其应用于训练和测试。所有预处理步骤（包括缺失值填补）都应仅从训练数据“学习”参数，以避免数据泄漏。在实现上，可先拆分数据，再对训练集拟合填充器，然后对训练和测试分别 transform。

```
median_val = df[col].iloc[:n_train].median()
# 只用训练集
```

2. 特征工程

2.1 特征工程合理性

本项目构造了许多新特征，如得房率（套内面积/建筑面积）、房屋户型拆分出的室厅卫数量、总楼层和楼层类别、多方向朝向独热、以及各类多标签特征。这些衍生特征大体上有业务意义，能丰富模型的信息表示。需要注意的是，有些衍生特征与原始特征存在高度相关性。例如得房率和套内面积/建筑面积本身强相关，总楼层与楼层类别也相关。线性模型中高度共线的特征可能造成不稳定，尤其是对于OLS而言。虽然正则化模型可以缓解，但最好仍是不要派生过多维数，并提前识别多重共线性。

代码还大量自动生成非线性特征。所有特征之间两两交互（如 面积 $\times$ 房龄，绿化率 $\div$ 容积率）；自身的平方项（如 面积²，房龄²）；导致高度相关的组合变量冗余存在，指数级放大共线性问题。

3. 创建交互项

```
[ ]: import pandas as pd
import numpy as np
from itertools import combinations, product
from pathlib import Path
def generate_nonlinear_features(df, verbose=True):
    """
    根据明确的规则生成非线性特征：
    1. 数值特征的平方项
    2. 数值特征之间的二元交互项
    3. 数值特征与二元(虚拟)特征的交互项

    Args:
        df (pd.DataFrame): 合并后的 (train + test) DataFrame.
        verbose (bool): 是否打印日志.

    Returns:
        pd.DataFrame: 增加了新特征的 DataFrame.
    """
    df_aug = df.copy()

    # 自动识别列类型
    # 假设二元/虚拟变量是那些只包含 0 和 1 的列
    binary_cols = [col for col in df.columns if set(df[col].unique()) == {0, 1}]
    numeric_cols = [col for col in df.select_dtypes(include=np.number)]
```

这导致了现在的模型维数过多。特征数超过300时，存在强共线性的概率高。项目生成了大量二元特征。二元特征本身之间经常高度相关，尤其是“互斥的多类变量”（如朝向、用途、楼层类型），若未去掉一个基准类，很容易线性相关；多热编码特征（如配套设施、建筑结构）可能共现（如空调+冰箱总是同时出现），也导致多重共线性。

Processing Price dataset: (103871, 233)

自动识别: 19 个数值特征, 214 个二元特征。

成功生成 190 个新的非线性特征。

Saved augmented Price data (train shape=(103871, 423), test shape=(34017, 423))

Processing Rent dataset: (98899, 196)

自动识别: 17 个数值特征, 179 个二元特征。

成功生成 153 个新的非线性特征。

Saved augmented Rent data (train shape=(98899, 349), test shape=(9773, 349))

建议: 在生成二次特征前先筛选出与目标相关度较高或有业务意义的特征进行组合, 而非穷举所有组合。也可以在生成后使用特征选择方法 (如基于Lasso系数、信息增益或VIF阈值) 筛除一批无用特征。也可考虑在线性模型中对高度相关的一组特征进行降维处理 (如主成分分析) 或选取其中信息量最大的一个使用, 防止共线性干扰模型参数。对树模型而言, 共线性影响不大, 可以保留。

2.2 特征使用

代码优先使用小区板块信息 ('板块_comm'), 再用房产板块信息 ('板块') 填充其缺失值, 最终构建出('板块_final')列。但该创建后并未使用, 后续均使用('区域')列、('板块')列来分层填充训练集与测试集各个列的空缺数据。('板块_comm')也未改为虚拟变量, 是否有遗漏?

3. 模型构建

3.1 Pipeline重复变换

目前线性模型Pipeline中已经对数据进行了StandardScaler, 但在预处理阶段其实已经对数值特征做了Z-score标准化 (调用了standardize_data)。对于随机森林Pipeline也包含了标准化和缺失值填充, 但预处理阶段几乎已经填充了所有缺失并且

数值无需标准化给树模型使用。这种重复变换虽然不影响结果 (再次标准化基本不会改变分布), 但增加了计算开销和代码复杂性。

e. Ridge模型

```
[ ]: from sklearn.linear_model import Ridge

def build_ridge_pipeline():
    return Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler()),
        ('model', Ridge()) # 注意: Ridge 通常不需要 max_iter
    ])
```

g.非线性模型（附）

```
: # --- Random Forest model cell ---  
from sklearn.ensemble import RandomForestRegressor  
  
def build_rf_pipeline():  
    return Pipeline([  
        ('imputer', SimpleImputer(strategy='median')),  
        ('scaler', StandardScaler()),  
        ('model', RandomForestRegressor(random_state=111))  
    ])  
  
rf_param_grid = {  
    'model__n_estimators': [100, 200],  
    'model__max_depth': [None, 10, 20],  
    'model__min_samples_leaf': [1, 2, 4]  
}
```

建议：精简Pipeline中的步骤，与前述预处理相协调。例如，如果预先对特征进行了标准化，就不需要再在Pipeline里重复此步骤；对于树模型，可简化Pipeline去掉不必要的scaler和imputer以加快训练速度。保持预处理和Pipeline一致，可以避免数据重复处理或潜在的不一致。

3.2 模型性能与改进

建议：可以考虑结合多个模型的优势，例如用Stacking或加权融合不同模型的预测，以提高稳健性。在确保不泄漏测试信息的前提下，可以用交叉验证预测的方法融合，以减小单一模型的偏差。